Saga Orchestration in Microservices

durable workflows, not cron jobs

Watch the 4-min video

In one line: an orchestrated saga runs a multi-service business process as local transactions with compensations, and a workflow engine (Temporal, Step Functions, Durable Functions) gives it durable state, retries, timeouts and crash recovery, so you don't hand-build a fragile state machine.

1 The orchestrator crashed. Was the card charged?

An order needs payment, stock and shipping, in three services with three databases (lesson 23), so there is no single ACID transaction. Halfway through, the orchestrator crashes. Did payment succeed? Retry it, or refund? Hand-written orchestrators keep progress in state tables, poll with cron jobs and grow fragile fast.

2 Saga recap



Figure 1. A saga: each step commits locally; if a later step fails, earlier steps are undone by compensating actions.

A **saga** is a sequence of local transactions, each with a **compensating action** that semantically undoes it (a refund, not a rollback). In **choreography** services react to each other's events; in **orchestration** one orchestrator tells each service what to do and decides what happens next (lesson 24). The happy path is easy. The hard part is crashes, timeouts and retries in the middle, and running compensations in reverse order.

3 Durable execution

A workflow engine turns that into a platform. You write the workflow as ordinary-looking code: charge, then reserve, then ship. The engine records every step's result in an **event history**. If a worker crashes, another worker **replays** the history: completed steps return their recorded results instead of running again, and execution continues exactly where it stopped. A workflow can sleep for days and still resume.

Activities (the calls to services) get declarative **retry policies** with exponential backoff and **timeouts**. If a step finally fails, the workflow runs compensations for the completed steps. You can **query** a running workflow for its current step, which helps support teams.

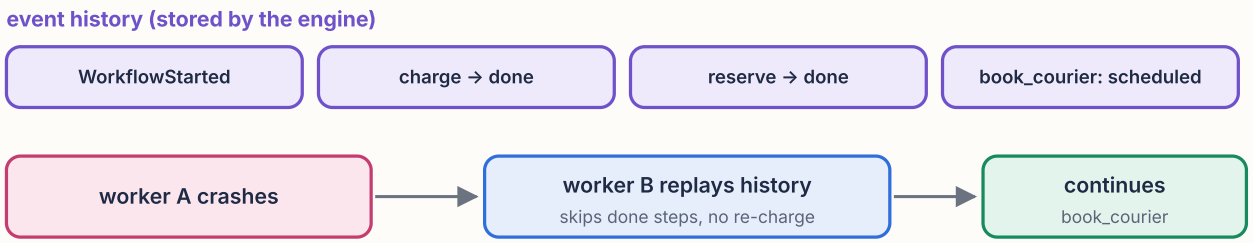


Figure 2. Replay: the history, not the worker's memory, is the source of truth.

```
@workflow.defn
class Fulfil:
    @workflow.run
    async def run(self, order):
        await workflow.execute_activity(charge, order, retry_policy=RETRY, start_to_close_timeout=T)
        try:
            await workflow.execute_activity(reserve, order, ...)
            await workflow.execute_activity(book_courier, order, ...)  # retried for up to 3 days
        except ActivityError:
            await workflow.execute_activity(release_stock, order, ...)   # compensations
            await workflow.execute_activity(refund, order, ...)
            raise
```

Figure 3. Temporal-style Python: the saga reads like normal code; the engine makes it durable.

	Hand-built orchestrator	Workflow engine
Progress after a crash	own state table, custom recovery code	replayed from stored history
Retries and timeouts	hand-coded, often inconsistent	declarative policy per activity
Waiting days	cron jobs polling tables	durable timers
Visibility	SQL queries on state tables	UI and queries per workflow

RULES AND TRADE-OFFS

- **Activities must be idempotent:** the engine may run them more than once (e.g. pass the order ID as an idempotency key to payment).
- **Workflow code must be deterministic:** no random numbers, direct clock reads or network calls inside it; use the engine's APIs, because replay must take the same path.
- **Versioning:** changing code while workflows are running needs explicit versioning or patching.
- **Another platform** to run or pay for, and keep business rules in the services, not all in the workflow.

WORKED EXAMPLE: ORDER FULFILMENT ON TEMPORAL

The workflow charges the card, reserves stock and books a courier, each as an activity with retries. The courier API is down; the activity retries with backoff for three days, then fails. The workflow releases the stock and refunds the payment, in reverse order. A worker restart on day two loses nothing, because the history is stored in Temporal's database; the new worker replays it and keeps waiting.

4 Key terms

Saga: sequence of local transactions with compensations

Orchestration: one coordinator directs the steps

Activity: a side-effecting step, retried by policy

Compensating action: business undo for a committed step

Durable execution: workflow state survives crashes via history

Deterministic workflow: same history replays to the same decisions

5 Interview questions

Q1. What is a saga? BASIC

A sequence of local transactions across services, each with a compensating action that undoes it if a later step fails.

Q2. Orchestration vs choreography? BASIC

Orchestration has one coordinator that commands each step and tracks state; choreography has services reacting to each other's events with no central controller.

Q3. Why use a workflow engine for sagas? SCENARIO

It provides durable state, retries with backoff, timeouts, crash recovery by replaying history, and compensation logic, instead of a fragile hand-built state machine.

Q4. The courier step fails after days of retries. What happens? SCENARIO

The workflow runs compensations for completed steps in reverse order: release stock, then refund the payment, and records the outcome.

Q5. Why must workflow code be deterministic? TRICKY

Recovery replays the history through the code; if the code makes different decisions (random values, clock reads), it diverges from the recorded history.

Q6. What do you need to watch out for? TRICKY

Idempotent activities, deterministic workflow code, and versioning for long-running workflows when the code changes.

Next: SD-L-017 · CQRS and Event Sourcing in Microservices backendroadmap.com